



LU1 Proof of Concept – Vrij lezen op maat

"Van mini-leesprofiel naar drie onderbouwde leesadviezen"

Je ontwikkelt, individueel, een Proof of Concept voor het project 'Vrij lezen op maat'. Hierin verwerk je de volgende functionele- en niet-functionele requirements.

Functionele Requirements

FR1 - Leesprofiel invullen

Als student wil ik een leesprofiel invullen, zodat de app passend leesmateriaal voor mij kan selecteren.

- De student kan vragen beantwoorden over taalniveau, genre, onderwerp, gewenste lengte en leesdoel.
- Verplichte velden zijn gemarkeerd; het profiel kan niet onvolledig worden opgeslagen. Wanneer ik (per ongeluk) weg navigeer wil ik niet opnieuw alle profielgegevens opnieuw in hoeven te vullen.
- Na opslaan krijgt de student een bevestiging.
- Er wordt een beperkte set vragen gehanteerd tijdens het Proof of Concept, kies een representatieve set (minimaal 3) uit de voorbeeldvragen.

FR2 - Leesprofiel inzien en wijzigen

Als student wil ik mijn opgeslagen leesprofiel kunnen bekijken en aanpassen, zodat mijn advies blijft kloppen als mijn voorkeuren veranderen.

- Een eerder opgeslagen profiel wordt correct getoond.
- Wijzigingen worden opgeslagen en gebruikt bij een volgend advies.
- Wanneer ik mijn leesprofiel heb aangepast, zonder deze op te slaan wil ik deze actie kunnen bevestigen of annuleren.

FR3 - Leesadvies ontvangen

Als student wil ik op basis van mijn profiel passende leessuggesties krijgen, zodat ik makkelijker een geschikt boek of tekst kies.

- Na het invullen toont de app minimaal 3 suggesties.
- Elke suggestie bevat titel, korte beschrijving en een motivatie ("waarom dit bij jou past").
- De suggesties komen uit de aanwezige database; er worden alleen bestaande titels getoond.



FR4 – Catalogus inzien

Als student wil ik de catalogus doorbladeren en filteren, zodat ik zelf gericht titels vind buiten het advies om.

- De catalogus toont titels gepagineerd met navigatie tussen pagina's; het totaal aantal resultaten is zichtbaar.
- De student kan filteren op relevante metadata (taalniveau, genre, onderwerp, lengte - spiegelt de profielvelden uit FR1).
- Filters zijn combineerbaar en resetbaar.
- Elke titel toont minimaal titel + korte beschrijving (consistent met FR3).

FR5 – Leeslijst bijhouden

Als student wil ik titels op een persoonlijke leeslijst zetten en markeren wat ik gelezen heb, zodat ik overzicht houd.

- De student voegt een titel toe vanuit de catalogus (FR5) of het advies (FR3).
- Elk item is te togglen tussen **niet-gelezen** en **gelezen**.
- De leeslijst toont per item de status.
- De leeslijst is persoonlijk; zichtbaar voor de eigenaar en de gekoppelde docent (gekoppeld aan FR6).

FR6 – Inzage in leeslijst door docenten

Als docent wil ik de leeslijsten inzien en aanvullen, zodat ik hun leesproces volg en bijstuur.

- De docent ziet de lijst van studenten die een leesprofiel hebben ingevuld.
- De student koppelt zichzelf aan een docent.
- De docent ziet uitsluitend de leeslijsten van gekoppelde studenten.
- De docent kan een item uit de catalogus toevoegen aan de leeslijst van een gekoppelde student.
- Toegang is rolgebaseerd afgedwongen in de back-end (NFR6).



Niet Functionele Requirements

NFR1 - Componentgebaseerde, consistente architectuur

Als ontwikkelaar wil ik werken met herbruikbare componenten binnen één consistente architectuur, zodat het team efficiënt kan samenwerken en de app uitbreidbaar blijft.

- De applicatie is opgebouwd uit herbruikbare componenten; gedeelde UI (bv. knoppen, formulervelden) wordt niet gedupliceerd.
- Er is één afgesproken architectuurpatroon (bv. duidelijke scheiding front-end / back-end / data laag) dat door de hele codebase consequent wordt toegepast.
- Een nieuwe feature van vergelijkbare omvang past binnen de bestaande structuur zonder de architectuur te doorbreken.

NFR2 - README met uitleg over de codebase

Als (nieuwe) ontwikkelaar wil ik een README die de codebase uitlegt, zodat ik snel productief ben.

- De README beschrijft hoe je het project lokaal opzet en draait.
- De README legt de mappenstructuur en de belangrijkste onderdelen uit.
- Een teamlid kan op basis van de README zonder mondelinge uitleg een eenvoudige wijziging doen en draaien.
- Bevat een verwijzing of uitleg van gekozen coding style guide. Dit is ook doorgevoerd in de code

NFR3 - Geautomatiseerde tests op de kernlogica

Als ontwikkelaar wil ik geautomatiseerde tests op de belangrijkste logica, zodat ik veilig kan wijzigen zonder bestaande functionaliteit te breken.

- De matching-/advieslogica is afgedekt met unit tests, inclusief minstens één rand-/foutgeval of belangrijke UI-componenten zijn afgedekt met component tests (gedrag bij invoer en weergave).

NFR4 - Responsive en zonder functieverlies

Als gebruiker wil ik de app op verschillende apparaten en browsers volledig kunnen gebruiken, zodat ik niet afhankelijk ben van één opstelling.

- De interface is responsive en bruikbaar op desktop en smartphone.
- Geen enkele kernfunctie (zie FR's) gaat verloren tussen desktop en mobiel.

NFR5 – Accessibility

Als gebruiker met een beperking wil ik de webapplicatie ook kunnen gebruiken.

- De front-end is WCAG level A compliant.



NFR6 – Authenticatie

Als gebruiker wil ik alleen toegang tot de gegevens die ik mag inzien of bewerken.

- De back-end is afgeschermd doormiddel van JWT-token authenticatie / autorisatie
- De front-end houdt rekening met paden waar de gebruiker geen toegang toe heeft en stuurt de gebruiker terug naar een ‘veilig’ pad zoals bijvoorbeeld de homepage of toont een duidelijke melding.

NFR7 - Passende dataopslag

Als ontwikkelaar wil ik elk datatype opslaan in de store die past bij vorm en toegangspatroon, zodat integriteit én flexibiliteit beide worden bediend.

- Transactionele, sterk gerelateerde gegevens staan in een relationele database met constraints/foreign keys.
- De contentcatalogus (titels met wisselende metadata) staat in een document store.